



Audit tecnico finale

Gestionale prenotazioni parrucchiere

DATA

10 luglio 2026

VERSIONE

1.0.0 — MVP

AUTORE

Audit tecnico

Stato complessivo: MVP funzionante in modalità DEMO locale; produzione Supabase predisposta ma non ancora attiva online. Completamento stimato: ~90% come demo, ~65% verso produzione reale, ~25% verso SaaS multi-salone.

Legenda:

COMPLETO

PARZIALE

DEMO

PREDISPOSTO

MANCANTE

CRITICO

PROD READY

1. Executive Summary

È stata costruita una web app gestionale per saloni di parrucchieri (React + Vite + TypeScript + Tailwind + Supabase): sito vetrina, autenticazione, prenotazione con calcolo disponibilità reale, area cliente e pannello admin completo (CRUD servizi, staff, disponibilità, clienti, calendario) e notifiche interne.

Concetto chiave: l'app ha **due modalità con switch automatico**.

- **DEMO (attuale, default):** senza credenziali Supabase l'app parte e usa un client “finto” che salva su `localStorage`. Login, prenotazioni, CRUD admin e notifiche **funzionano davvero**, ma i dati vivono **solo nel browser**.
- **PRODUZIONE (Supabase):** con un `.env` valido la stessa app usa Supabase reale (Auth, DB, RLS, realtime). Codice e schema ci sono, **ma nessun progetto Supabase è collegato o testato online**.

Funziona al 100% (demo): vetrina, login/ruoli, wizard prenotazione con slot reali, anti-overlap, cancellazione con soglia, notifiche in-app, tutti i CRUD admin, statistiche. **Demo:** persistenza (localStorage), auth demo. **Predisposto non online:** intero backend Supabase. **Manca:** email/SMS/WhatsApp, Edge Functions, reminder, multi-salone, deploy, pagine legali.

Giudizio: ottimo prototipo/MVP dimostrabile e buona base di prodotto. Non è ancora un gestionale di produzione: per venderlo a un salone reale serve collegare Supabase, aggiungere email transazionali e fare il deploy.

2. Stato complessivo progetto

Area	Stato	Note	Priorità
Frontend	COMPLETO	15 pagine presenti e navigabili, responsive	—
Backend	PREDISPOSTO	Solo Supabase (BaaS); nessuna Edge Function/email	Critico
Database	PREDISPOSTO	<code>schema.sql</code> completo ma non eseguito online	Critico
Auth	PARZIALE	Supabase Auth pronta; demo = localStorage in chiaro	Critico
Prenotazioni	COMPLETO	Logica robusta client + vincolo DB anti-overlap	Importante
Admin	COMPLETO	CRUD servizi/staff/disponibilità/clienti/calendario	—
Notifiche	PARZIALE	Solo interne (in-app). Nessuna email/SMS	Importante
Demo mode	COMPLETO	Funziona end-to-end su localStorage	—
Produzione Supabase	PREDISPOSTO	Mai collegata/testata online	Critico
Sicurezza	PARZIALE	RLS ottima in Supabase; demo manipolabile lato client	Critico
Performance	PARZIALE	Bundle unico 626 KB (176 KB gzip), no code-splitting	Migliorabile
UX/UI	COMPLETO	Design premium, coerente, responsive	—
Deploy	MANCANTE	Nessun hosting/ dominio	Critico

3. Architettura attuale

55 file. Struttura modulare per feature.

```
src/
  components/(ui/) Button, Card, Modal, Input, Select, Badge, Calendar, ServiceCard,
                  StaffCard, AppointmentCard, Navbar, Footer, NotificationBell...
  config/          salonConfig.ts (personalizzazione + palette)
  context/         AuthContext.tsx
  hooks/           useServices, useStaff, useNotifications, useClientAppointments
  layouts/         PublicLayout, DashboardLayout, AdminLayout
  lib/             env, supabase(switch), localClient(mock), demoData,
                  appointments(data layer), scheduling
  pages/           Home, Services, Booking, Login, Register, NotFound
    client/        Dashboard, Appointments, Notifications
    admin/         Dashboard, Calendar, Services, Staff, Clients, Settings
  routes/          ProtectedRoute, AdminRoute, AppRoutes
  types/ utils/    database.ts · availability(slot), format
supabase/schema.sql schema + RLS + trigger + seed
```

Flusso dati (switch automatico)

```
React UI (pages)
  → Hooks + lib/appointments + lib/scheduling
  → lib/supabase.ts → isSupabaseConfigured ?
    NO (demo) → localClient (mock) → localStorage
    SI (prod) → @supabase/supabase-js → Supabase (Postgres+Auth+Realtime+RLS)
```

Nota onesta: l'astrazione dati non è un `dataProvider` separato, bensì un **client Supabase intercambiabile**. Funziona in entrambe le modalità, ma diverse pagine admin e la Login **chiamano supabase direttamente** invece di passare da hook/lib comuni.

4. Analisi Database

Fonte: `supabase/schema.sql` — completo e ben scritto, ma **fornito, non eseguito online**. In demo le tabelle sono array in `localStorage`.

Tabella	Presente	Completa	Modalità	Note
profiles	Si	Si	PRED / DEMO	FK auth.users, ruolo, trigger creazione
services	Si	Si	PRED / DEMO	durata, prezzo, categoria, active
staff_members	Si	Si	PRED / DEMO	ruolo, bio, avatar, active
staff_availability	Si	Si	PRED / DEMO	orari settimanali
salon_closures	Si	Si	PRED / DEMO	chiusure/ferie
appointments	Si	Si	PRED / DEMO	status, prezzo, exclusion constraint anti-overlap
notifications	Si	Si	PRED / DEMO	generate da trigger

Pronto nel file: enum, FK, indici, `updated_at` auto, **RLS su tutte le tabelle**, trigger creazione profilo e notifiche, **exclusion constraint GiST** anti doppia-prenotazione, seed. **Solo predisposto:** esecuzione su Supabase reale. **Manca:** progetto creato, schema online, admin promosso, dati demo (oggi solo localStorage).

5. Analisi Backend

Backend interamente **Supabase (BaaS)**: nessun server applicativo custom.

Componente	Stato	Note
Supabase Auth	PREDISPOSTO	Email/password, sessione persistente, reset (solo modalità Supabase)
Database	PREDISPOSTO	Schema completo, non eseguito online
RLS	PREDISPOSTO	Cliente vede i propri, admin tutto, scrittura solo admin
Funzioni SQL	PREDISPOSTO	is_admin, handle_new_user, notify_appointment, set_updated_at
Realtime	PARZIALE	Solo notifiche; in demo simulato
Anti-overlap	COMPLETO	Doppio livello: client + exclusion constraint DB
Edge Functions	MANCANTE	Nessuna funzione serverless
Email transazionali	MANCANTE	Nessun invio email
Webhook	MANCANTE	—
Validazioni server	PARZIALE	Vincoli DB + RLS; nessuna logica server

6. Analisi Frontend

Pagina	Stato	Problemi	Prio.
Home	COMPLETO	Immagini Unsplash esterne	—
Servizi	COMPLETO	—	—
Prenota	COMPLETO	Timezone browser	Imp.
Login	COMPLETO	Reset reale solo Supabase	—
Registrazione	COMPLETO	—	—
Dashboard Cliente	COMPLETO	—	—
Appuntamenti Cliente	COMPLETO	—	—
Notifiche	COMPLETO	—	—
Admin Dashboard	COMPLETO	Ricavi su 7gg fissi	—
Admin Calendario	COMPLETO	Solo vista giornaliera	Migl.
Admin Servizi	COMPLETO	—	—
Admin Staff	COMPLETO	—	—
Admin Clienti	COMPLETO	Sola lettura	—
Admin Disponibilità	COMPLETO	—	—
Not Found	COMPLETO	—	—

Componenti riutilizzabili coerenti, loading/empty state ovunque, form validati (RHF + Zod), palette da config. Accessibilità base presente (label, aria-label), non auditata WCAG.

7. Analisi Prenotazioni

Robusto

- Servizio (durata+prezzo), staff o **"qualsiasi disponibile"** (primo libero).
- Slot su durata reale con **passo** e **buffer** configurabili.
- Rispetto orari staff, **giorni chiusi/ferie, no passato, no sovrapposizioni**.
- **Cancellazione** entro soglia configurabile (default 3h).
- **Doppia barriera anti-overlap**: client + exclusion constraint DB (verificato: secondo slot rifiutato, err `23P01`).

Fragile / casi limite

- **Timezone/DST**: calcoli su ora locale del browser → possibili disallineamenti. Da normalizzare a timezone salone.
- **Race condition** stesso slot: gestita solo dal vincolo DB; il perdente riceve errore e risSelected.
- Nessun "hold" temporaneo dello slot durante la compilazione.

8. Analisi Admin Dashboard

- **Statistiche**: oggi, settimana, ricavi (7gg), servizio top, cancellazioni — funzionanti, finestra fissa 7 giorni.
- **Calendario**: vista giornaliera, filtri staff/stato, dettaglio, creazione manuale, completa/cancella. Manca vista settimana/mese grafica.
- **CRUD Servizi/Staff**: crea/modifica/disattiva; eliminazione con fallback a disattivazione se ci sono appuntamenti.
- **Clienti**: lista + ricerca + conteggio (sola lettura).
- **Disponibilità**: fasce settimanali per stylist + chiusure/ferie.

Mancano: export (CSV/PDF), audit log, modifica orario di un appuntamento esistente, permessi staff granulari.

9. Analisi Demo Mode DEMO

Cos'è: senza credenziali valide, `lib/supabase.ts` usa `localClient`, un mock dell'interfaccia Supabase che persiste su `localStorage`.

Salva in localStorage: `bns_demo_db_v1` (tutte le tabelle), `bns_demo_users_v1` (utenti, **password in chiaro**), `bns_demo_session_v1` (sessione).

Credenziali: Admin `admin@demo.it / admin1234` · Cliente `cliente@demo.it / demo1234`.

Provabile: login, navigazione, prenotazione con slot reali, cancellazione, notifiche in-app, tutti i CRUD admin, statistiche. I dati persistono tra refresh.

NON è reale: dati solo in quel browser (non condivisi), auth finta, nessuna sicurezza, realtime simulato.

Rischio: se scambiata per produzione dà falsa impressione di app "già online". Il badge **"Demo"** in navbar mitiga; va comunque comunicato al cliente.

10. Supabase Production Mode PREDISPOSTO

Con `.env` valido l'app usa Supabase reale. **Da fare (non ancora fatto)**:

- Creare progetto Supabase.
- Eseguire tutto `supabase/schema.sql`.
- Compilare `.env` (URL + anon key).
- Registrarsi e promuovere il primo admin via SQL.
- Configurare/disattivare conferma email.

Possibili errori: trigger `handle_new_user` non attivo → profilo assente; conferma email attiva blocca il primo login; policy `is_admin()` richiedono `role='admin'` già impostato.

Checklist: progetto ☐ · schema ☐ · `.env` ☐ · admin ☐ · email ☐ · test cliente/admin ☐.

11. Sicurezza

Aspetto	Modalità Supabase	Modalità Demo
Ruoli	Solido (role + is_admin RLS)	Solo lato client
Protezione rotte	Protected/AdminRoute (UX)	Aggirabile
RLS dati	Completa	Assente (localStorage)
Dati cliente	Isolati da RLS	Tutti leggibili nel browser
Manipolazione client	Irrilevante (server valida)	Totale

In produzione Supabase la sicurezza è **buona** (le protezioni vere sono in RLS). La **demo non è sicura per definizione**. Password demo in chiaro. Nessun rate limiting/captcha su login/registrazione.

12. Performance ed efficienza

- **Bundle:** singolo chunk JS **626 KB (~176 KB gzip)** + CSS ~30 KB. Warning "chunk > 500 KB". Nessun code-splitting per rotta.
- **Dati:** query mirate con indici in produzione; disponibilità = 3 fetch parallele/giorno. Nessuna cache client (React Query assente).
- **localStorage (demo):** R/W sincrona dell'intero DB a ogni operazione — ok per dati piccoli, non scala.
- **Ottimizzazioni:** lazy-load rotte (React.lazy), `manualChunks`, React Query, immagini locali.

13. Bug, rischi e criticità

Criticità	Gravità	Impatto	Soluzione
Nessun progetto Supabase collegato/testato	CRITICO	L'app "vera" non è online	Creare progetto, eseguire schema, testare
Demo scambiabile per produzione	CRITICO	Falsa percezione	Comunicazione + badge
Password demo in chiaro	Alta	Cattiva pratica se estesa	Non usare demo come auth reale
Timezone/DST negli slot	Alta	Slot errati in fusi/cambio ora	Timezone salone
Script <code>typecheck</code> assente	Media	Comando citato ma mancante	Aggiungere <code>"typecheck": "tsc -b"</code>
Chiamate <code>supabase</code> dirette in 6 pagine	Media	Astrazione non piena	Spostare in hook/lib
Bundle unico > 500 KB	Media	Primo load più lento	Code-splitting
Nessuna email	Alta	Funzioni chiave assenti	SMTP/Resend + Edge Fn
Nessun test automatico	Media	Regressioni non rilevate	Unit/e2e

Verifica reale: `tsc -b` → **PASSATO** (0 errori) · `vite build` → **PASSATO** (solo warning chunk) · nessun `TODO/FIXME`.

14. Cosa manca totalmente

Non esiste: Edge Functions · email transazionali · SMS/WhatsApp · reminder automatici · cancellazione via link · Google Calendar · export appuntamenti · audit log · test automatici · monitoring (Sentry) · analytics.

Per produzione: progetto Supabase attivo · schema online · deploy/hosting · dominio · conferme email · timezone.

Per vendibilità reale: pagine legali (privacy/cookie/GDPR) · consenso dati · backup · onboarding salone.

Per SaaS multi-salone: multi-tenant · isolamento dati per salone · billing · super-admin · provisioning.

15. Roadmap consigliata

Fase	Attività	Priorità	Difficoltà	Tempo	Impatto
1 — Produzione reale	Supabase, schema, .env, admin, test RLS/prenotazioni, script typecheck	Critico	Bassa	1–2 gg	Alto
2 — Hardening sicurezza	Policy, timezone, rate limiting/captcha, chiamate→lib, validazioni	Critico	Media	3–5 gg	Alto
3 — Notifiche email/SMS	Edge Fn su eventi, SMTP/Resend conferme+reminder, cancella via link, WhatsApp	Importante	Media	4–7 gg	Alto
4 — Deploy & dominio	Vercel, env, dominio, immagini locali, GDPR, analytics+Sentry	Importante	Bassa	1–2 gg	Alto
5 — Multi-salone / SaaS	Multi-tenant + RLS, super-admin, billing, onboarding	Migliorabile	Alta	3–6 sett.	Strategico
6 — Prodotto BNS	Calendario settimana/mese, export, audit, permessi, Google Cal, PWA	Migliorabile	Alta	3–6 sett.	Alto

16. Checklist finale test

Demo

- ☐ `npm install` → `npm run dev` senza `.env` · ☐ badge "Demo" · ☐ login admin/cliente
- ☐ Prenotazione → appare + notifica · ☐ doppio slot rifiutato · ☐ cancellazione + notifica
- ☐ CRUD servizi/staff/disponibilità · ☐ refresh → dati persistono

Supabase (produzione)

- ☐ Schema eseguito · ☐ `.env` valido (badge sparisce) · ☐ registrazione → profilo (trigger)
- ☐ Admin via SQL → `/admin` · ☐ cliente non vede altrui (RLS) · ☐ overlap constraint · ☐ realtime

Trasversali

- ☐ Responsive mobile/tablet/desktop · ☐ `npm run build` pulito · ☐ deploy raggiungibile

17. Giudizio finale

Stato reale: MVP completo e verificato in **modalità DEMO**; produzione Supabase **predisposta ma non attivata online**. Frontend e logica di prenotazione di buona qualità; backend pronto come schema/policy ma non collegato.

Completamento: ~90% come demo · ~65% verso produzione reale · ~25% verso SaaS multi-salone.

Scenario di vendita	Verdetto
Demo / prototipo di vendita	Sì, subito
Template / MVP custom per UN salone	Sì dopo Fase 1+3+4 (~1–2 settimane)
Gestionale di produzione affidabile	NON ANCORA
SaaS multi-salone	NO (serve Fase 5)

Al cliente: "È un MVP funzionante e personalizzabile; la demo mostra tutte le funzioni. Per andare online servono database, email di conferma e deploy." **Non promettere:** che è già online e sicuro, reminder automatici, multi-salone, GDPR pronto.

Prossima azione: creare il progetto Supabase, eseguire `schema.sql`, collegare `.env` e ripetere la checklist di produzione end-to-end.

Appendice — Esiti comandi reali (10/07/2026)

Comando	Esito	Note
npm install	OK	156 pacchetti, 0 vulnerabilità
npm run typecheck	N/D	Script assente; typecheck dentro build
tsc -b (typecheck)	PASSATO	0 errori, 0 warning TS
npm run build	PASSATO	Warning chunk > 500 KB (626/176 gzip)
Ricerca TODO/FIXME	NESSUNO	—
supabase diretto in pagine	6 file	Login, Admin Servizi/Staff/Settings/Clienti/Calendario